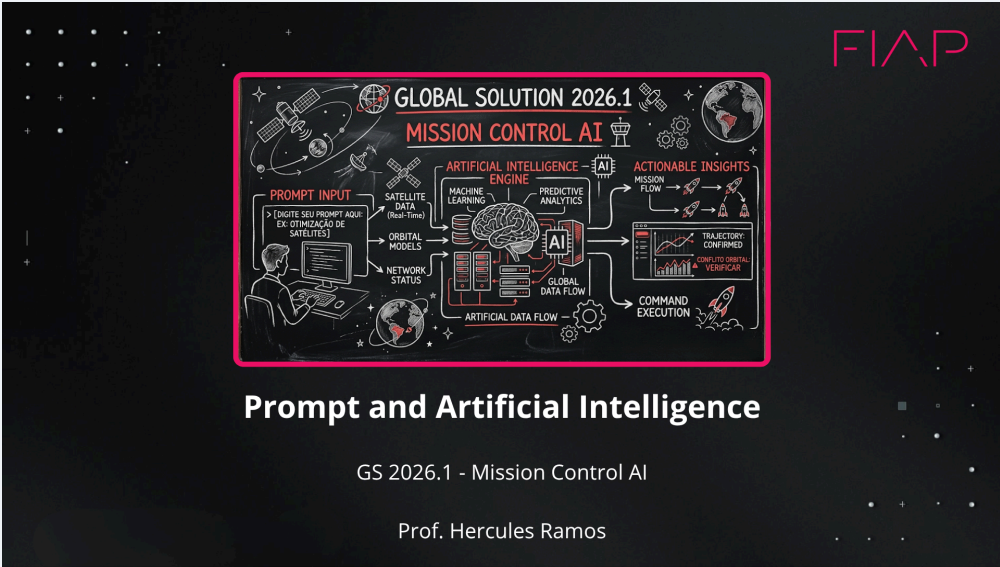


Global Solution 2026.1 - Prompt and Artificial Intelligence



Ciência da Computação - 1º Ano

 **Prompt and Artificial Intelligence**

A era da inteligência artificial está transformando todos os setores da economia. Nesta disciplina, você entenderá como prompts e modelos de linguagem interagem para criar soluções inteligentes. Exploraremos ferramentas como OpenAI e GPT, além de integrar essas tecnologias em plataformas práticas para criar chatbots e assistentes virtuais que respondem a comandos complexos.

FIAP - Global Solution 2026.1

Disciplina: Prompt and Artificial Intelligence

Período: 1º Semestre de 2026

Peso na Nota: **6 pontos** | Avaliação: **0 a 10**

 **Divulgação:** 25/05/2026;  **Entrega final:** 09/06/2026 até as 23h55

Contexto Geral

O **Global Solution – 1º Semestre de 2026** desafia os estudantes a desenvolver soluções tecnológicas inspiradas nos desafios da **indústria espacial moderna**. A proposta é transformar problemas relacionados à exploração espacial, sobrevivência fora da Terra, comunicação interplanetária, automação autônoma, sustentabilidade e segurança aeroespacial em oportunidades de inovação, impacto científico e avanço tecnológico.

Desafio da Turma — Mission Control AI

Neste desafio, os alunos deverão desenvolver um **sistema inteligente de monitoramento para controle básico de uma missão espacial experimental**. A solução deverá receber, interpretar e exibir dados simulados relacionados às condições operacionais da missão, incluindo:

-  **Temperatura** dos módulos
-  **Comunicação** (status do sinal)
-  **Energia** (nível de bateria / geração solar)

-  **Status dos módulos** da operação

As equipes deverão aplicar conceitos de **programação, algoritmos, pensamento computacional e inteligência artificial introdutória** para construção de uma plataforma funcional de análise operacional.

O sistema deve contemplar:

- Geração automática de **alertas** diante de parâmetros críticos
- **Organização** e estruturação das informações monitoradas
- **Visualização simplificada** dos dados da missão
- Estruturas lógicas para **tomada de decisão básica**
- **Respostas automatizadas** diante de situações críticas simuladas
- Uso de **IA generativa** para geração de dados simulados e/ou previsões

 O projeto deverá priorizar organização computacional, eficiência do código e clareza na representação das informações operacionais.

 Entregável Esperado — Solução com IA

Baseando-se nos conceitos estudados ao longo do semestre, cada equipe deverá criar **uma solução com inteligência artificial** que:

1. **Gere dados simulados** de uma missão espacial (temperatura, energia, comunicação, etc.)
2. **Exiba previsões ou análises** a partir desses dados
3. Apresente os resultados de forma clara e funcional

A solução pode ser implementada como:

Formato	Descrição
 Chatbot	Interface conversacional que interpreta comandos e responde com dados/alertas da missão
 Interface Web/App	Painel visual (dashboard) com dados em tempo simulado e respostas da IA
 Solução Visual	Representação gráfica dos dados com análise automatizada por IA

A forma de implementação é livre, desde que demonstre o uso de **IA integrada à lógica da missão**.

 Ferramentas Recomendadas

 Ambiente — Google Colab

O projeto pode ser feito **100% no Google Colab**, sem instalar nada no computador. É só abrir o notebook, escrever o código e executar célula por célula.

 Entreguem o link do notebook no GitHub com acesso público habilitado.

 Modelo de IA — Llama via Ollama no Colab

O modelo recomendado é o **Llama 3.1 1B**, rodando diretamente no Colab via **Ollama** — sem necessidade de criar conta ou chave de API.

Como instalar e usar no Colab (conforme visto em aula):

```
1 # 1. Instalar o Ollama no Colab
2 !curl -fsSL https://ollama.com/install.sh | sh
3
4 # 2. Iniciar o servidor em background
5 import subprocess, time
```

```

6 subprocess.Popen(["ollama", "serve"])
7 time.sleep(3)
8
9 # 3. Baixar o modelo Llama 3.1 1B
10 !ollama pull llama3.2:1b
11
12 # 4. Usar o modelo via biblioteca ollama
13 !pip install ollama -q
14
15 import ollama
16
17 resposta = ollama.chat(
18     model="llama3.2:1b",
19     messages=[
20         {
21             "role": "system",
22             "content": "Você é um sistema de controle de missão espacial. Analise os dados e indique alertas."
23         },
24         {
25             "role": "user",
26             "content": "Temperatura: 95°C | Energia: 18% | Comunicação: instável. Avalie o status."
27         }
28     ]
29 )
30
31 print(resposta["message"]["content"])

```

💡 Este é o ponto de partida. A partir daqui, vocês adicionam a lógica da missão, os alertas e a exibição dos dados.

Interface — Python (Recomendado)

A interface deve ser feita em **Python**, no próprio Colab ou como script local. Não é obrigatório criar uma interface web ou mobile — uma saída organizada no terminal ou no Colab já é suficiente para a nota máxima.

Outras interfaces (web, app, dashboard) são **opcionais** e podem ser usadas se o grupo já tiver familiaridade, mas **não são exigidas** e não somam pontos extras.

Exemplo simples de saída em Python:

```

1 def exibir_status(temperatura, energia, comunicacao, analise_ia):
2     print("=" * 45)
3     print(" 🚀 MISSION CONTROL – STATUS ATUAL")
4     print("=" * 45)
5     print(f" 🌡️ Temperatura : {temperatura}°C")
6     print(f" ⚡ Energia      : {energia}%")
7     print(f" 📡 Comunicação : {comunicacao}")
8     print("-" * 45)
9     print(" 🖥️ Análise da IA:")
10    print(f"   {analise_ia}")
11    print("=" * 45)
12
13 exibir_status(95, 18, "instável", resposta["message"]["content"])

```

Formação das Equipes

Modalidade	Permitido?
Individual	✔ Sim
Dupla	✔ Sim
Trio	✔ Sim
Grupos maiores	✘ Não

O Que Deve Ser Entregue

A entrega deve ser feita pelo portal da instituição, em um **arquivo .txt** (zipado se necessário) contendo as informações abaixo.

Conteúdo do arquivo .txt :

```
1 Nome completo e RM de todos os integrantes:
2 - [Nome] – RM: [número]
3 - [Nome] – RM: [número]
4 - [Nome] – RM: [número] (se trio)
5
6 Link do repositório GitHub: https://github.com/...
7
8 Link do vídeo de demonstração: https://...
```

 O vídeo deve ter **no máximo 3 minutos** e demonstrar a **funcionalidade completa** da solução.

✔ Checklist para Nota Máxima

Use este checklist antes de submeter seu projeto. Para tirar nota máxima, **todos os itens devem estar marcados**.

Entrega

- ☐ Arquivo **.txt** com nomes e RMs de todos os integrantes
- ☐ Link do GitHub no arquivo **.txt**
- ☐ Link do vídeo no arquivo **.txt**
- ☐ Vídeo com até **3 minutos** de duração
- ☐ Arquivo entregue no portal dentro do prazo

Repositório GitHub

- ☐ Repositório público e acessível
- ☐ Notebook **.ipynb** ou código Python no repositório
- ☐ README com as seções obrigatórias (ver seção abaixo)
- ☐ Pelo menos 2 prints do sistema funcionando no README

Solução com IA

- ☐ Modelo de linguagem integrado ao código e funcionando
- ☐ **System prompt** com contexto da missão espacial definido
- ☐ Resposta da IA exibida de forma legível no output

Funcionalidades da Missão

- ☐ Monitoramento de pelo menos **3 parâmetros** (ex: temperatura, energia, comunicação)
- ☐ Geração de **alertas automáticos** quando parâmetros estão fora do normal
- ☐ Lógica de **tomada de decisão** (ex: "Se energia < 20%, ativar modo de economia")
- ☐ **Resposta automatizada** para pelo menos uma situação crítica simulada

- ☐ Dados da missão organizados e visualmente compreensíveis

 Vídeo de Demonstração

- ☐ Apresenta os integrantes do grupo no início
- ☐ Demonstra todas as funcionalidades implementadas
- ☐ Mostra a IA gerando dados/respostas em tempo real
- ☐ Demonstra pelo menos um cenário de alerta ou situação crítica
- ☐ Áudio e imagem com qualidade suficiente para avaliação

 Critérios de Avaliação

 A avaliação leva em conta o **esforço, a criatividade e o aprendizado** demonstrado. Não esperamos um sistema perfeito — esperamos que vocês apliquem o que foi visto em aula com dedicação.

#	O que será avaliado	Pontuação
1	Entrega correta — <code>.txt</code> com nomes/RMs, link do GitHub e link do vídeo	10 pts
2	README completo — descrição, prints, instruções e link do vídeo	20 pts
3	Uso de IA — modelo de linguagem integrado e respondendo com contexto da missão	30 pts
4	Funcionalidades básicas — dados simulados exibidos + pelo menos um alerta ou decisão lógica	30 pts
5	Vídeo de demonstração — mostra o sistema funcionando, dentro de 3 minutos	10 pts
	TOTAL	100 pts

Como cada critério é avaliado

1. Entrega (10 pts)

O arquivo `.txt` está correto, com todos os dados e links funcionando no dia da correção.

2. README (20 pts)

Tem descrição do projeto, pelo menos 4 prints reais do sistema rodando, lista de tecnologias e instruções de como executar. README vazio ou copiado = desconto direto.

3. Uso de IA (30 pts)

O projeto deve usar algum modelo de linguagem integrado ao código, com um prompt que faça sentido para o contexto da missão espacial. A ferramenta é livre — a recomendação é o **Llama via Ollama no Colab** (sem necessidade de conta ou chave), mas outras opções como Groq, Gemini ou OpenAI também são aceitas.

Situação	Pontos
IA integrada, respondendo, prompt com contexto da missão	25–30
IA integrada mas prompt genérico ou sem relação com a missão	15–24
Tentativa de integração com erros ou sem retorno visível	5–14
IA não integrada	0–4

4. Funcionalidades (30 pts)

O sistema exibe dados simulados e tem pelo menos uma lógica de alerta ou decisão funcionando. Não precisa ser complexo — precisa funcionar.

Situação	Pontos
Dados simulados + alertas + resposta da IA exibida claramente	25–30
Dados simulados funcionando mas sem alerta ou decisão	15–24
Apenas exibição estática, sem lógica ou IA visível	5–14
Nada funcional	0–4

5. Vídeo (10 pts)

Mostra o grupo (nome ou rosto), executa o sistema ao vivo e demonstra pelo menos um cenário de alerta. Qualidade de câmera não é critério — clareza e completude sim.

 Como Deve Ser o README do Projeto

O README é obrigatório e deve estar na raiz do repositório (`README.md`). Não precisa ser longo — precisa ter as informações essenciais abaixo.

O que o README deve ter:

1. Nome do projeto e integrantes

```
1 # 🚀 Mission Control AI - [Nome do Projeto]
2
3 **Integrantes:**
4 - Nome Completo - RM: XXXXXX
5 - Nome Completo - RM: XXXXXX
```

2. O que o projeto faz

Escreva 2 ou 3 frases explicando o que o sistema faz e como a IA está envolvida.

Exemplo: *"Sistema de monitoramento de missão espacial em Python. Usa o modelo Llama via Ollama para analisar dados simulados de temperatura, energia e comunicação, e gera alertas automáticos quando a missão entra em situação crítica."*

3. Prints do sistema funcionando

Inclua pelo menos **2 prints reais** da saída do sistema — um mostrando os dados normais e um mostrando um alerta ou resposta da IA. Salve as imagens em uma pasta `assets/` e reference assim:

```
1 ## 🖼️ Demonstração
2
3 ![Dados da missão](assets/dados_missao.png)
4 ![Alerta crítico](assets/alerta_critico.png)
```

Os prints precisam ser do sistema **rodando de verdade**. Print de código parado não conta.

4. Como executar

Basta indicar que o projeto roda no Google Colab e linkar o notebook:

```
1 ## ▶️ Como Executar
2
3 Abra o notebook no Google Colab:
4 [Acessar Notebook](https://colab.research.google.com/drive/SEU_LINK_AQUI)
5
6 Execute as células em ordem. O modelo Llama será instalado automaticamente.
```

5. Link do vídeo

```
1 ## 🎥 Vídeo de Demonstração
2 [Assistir ao vídeo](https://link-do-video.com)
```

✅ Checklist do README

- ☐ Nome do projeto e integrantes com RM
- ☐ Descrição curta do que o sistema faz
- ☐ Pelo menos 2 prints reais do sistema rodando
- ☐ Instruções de como executar (link do Colab)
- ☐ Link do vídeo de demonstração

Situação	Penalização
Plágio ou cópia de outro grupo	Nota zero para todos os envolvidos
Uso de projeto pronto sem adaptação	Nota zero
Entrega fora do prazo (09/06 após 23h55)	-20 pts por dia de atraso
Vídeo acima de 3 minutos	-5 pts
README ausente ou em branco	-10 pts
Prints ausentes no README	-10 pts
Repositório privado ou inacessível na data de avaliação	-15 pts

📅 Cronograma

Evento	Data	Horário
🚀 Divulgação do desafio	25/05/2026	08h00
🕒 Entrega final	09/06/2026	até 23h55

❌ Não serão aceitas entregas fora do prazo sem justificativa prévia aprovada pelo professor.

💡 Dicas para uma Boa Entrega

- 🧪 Use o Google Colab — não precisa instalar nada, o modelo Llama sobe direto pelo Ollama
- 📷 Tire prints enquanto testa — não deixe para o último momento
- 🎥 Grave o vídeo com o sistema rodando ao vivo — não precisa ser editado, basta mostrar funcionando
- 📄 Capriche no README — ele vale 20 pontos e é o cartão de visitas do projeto
- ✅ Revise o checklist antes de submeter — muitos pontos são perdidos por esquecer itens simples

Boa sorte, futuros engenheiros do espaço! 🚀

📞 Contato

📧 profhercules.ramos@fiap.com.br

